UNIVERSIDAD
NACIONAL
DE COLOMBIA

Laboratory 2

Architectural Modeling

Jeisson Andrés Vergara Vargas

Large-Scale Software Architecture

2026-I

---

1. Objective

The objective of this lab is to apply the Model-Driven Software Engineering (MDE) paradigm in order to automate the generation of the skeleton of a software system from an architectural perspective. Students will create a computer program to define a DSL-based metamodel, define a set of transformation rules, define a model (as a metamodel instance), and execute the generation process.

In this laboratory, the **European Rail Traffic Management System (ERTMS)** — studied in Lab 1 — serves as the system to be modeled. The DSL is extended to represent its five architectural tiers (T_1 through T_5) and the nature of connectors that operate within a System of Systems.

DSL: Domain-Specific Language.

---

2. Prerequisites

A computer with Docker installed.

---

3. Instructions

3.1. Metamodeling

a. Create an **arch.tx** file to define the grammar of the DSL. The grammar is extended to cover the five architectural tiers of the ERTMS and the connector types that operate between them:

```
Model:
    'architecture' ':'
        elements*=Element
```

```
;

Element:
    Component | Connector
;

Component:
    'component' tier=TierType type=ComponentType name=ID
;

Connector:
    'connector' type=ConnectorType from=[Component] '->' to=[Component]
;

TierType:
    'presentation' | 'communication' | 'logic' | 'data' | 'physical'
;

ComponentType:
    'web_interface'   |
    'operator_ui'     |
    'driver_ui'       |
    'api_gateway'     |
    'microservice'    |
    'authority_service' |
    'database'        |
    'data_lake'       |
    'onboard_unit'    |
    'sensor'          |
    'actuator'        |
    'balise'
;

ConnectorType:
    'data_flow'         |
    'control_command'   |
    'event_notification'|
    'dependency'
;
```

b. Create a **metamodel.py** file to instantiate the metamodel from the DSL grammar:

```python
import os
from textx import metamodel_from_file

def create_metamodel():
    grammar = os.path.join(os.path.dirname(__file__), 'arch.tx')
    return metamodel_from_file(grammar)
```

3.2. Transformations

Create a **transformations.py** file. Each generator produces a skeleton artifact representative of the component's tier and type within the ERTMS architecture.

```
import os
import textwrap

# ──────────────────────────────────────────────
# T1 — Presentation
# ──────────────────────────────────────────────

def generate_web_interface(name):
    path = f'skeleton/{name}'
    os.makedirs(path, exist_ok=True)
    with open(os.path.join(path, 'index.html'), 'w') as f:
        f.write(textwrap.dedent(f"""
            <!DOCTYPE html>
            <html>
            <head><title>{name} — Passenger Portal</title></head>
            <body>
                <h1>ERTMS Passenger Portal</h1>
                <p>Component: {name}</p>
            </body>
            </html>
        """))
    with open(os.path.join(path, 'Dockerfile'), 'w') as f:
        f.write(textwrap.dedent("""
            FROM nginx:alpine
            COPY index.html /usr/share/nginx/html/index.html
        """))

def generate_operator_ui(name):
    path = f'skeleton/{name}'
    os.makedirs(path, exist_ok=True)
    with open(os.path.join(path, 'dashboard.html'), 'w') as f:
        f.write(textwrap.dedent(f"""
            <!DOCTYPE html>
            <html>
            <head><title>{name} — Operator Dashboard</title></head>
            <body>
                <h1>ERTMS Operator Dashboard</h1>
                <p>Component: {name}</p>
                <p>Status: Monitoring active routes...</p>
            </body>
            </html>
        """))
    with open(os.path.join(path, 'Dockerfile'), 'w') as f:
        f.write(textwrap.dedent("""
            FROM nginx:alpine
            COPY dashboard.html /usr/share/nginx/html/index.html
        """))
```

```
def generate_driver_ui(name):
    path = f'skeleton/{name}'
    os.makedirs(path, exist_ok=True)
    with open(os.path.join(path, 'driver.html'), 'w') as f:
        f.write(textwrap.dedent(f"""
            <!DOCTYPE html>
            <html>
            <head><title>{name} — Driver Interface</title></head>
            <body>
                <h1>ERTMS Driver Interface</h1>
                <p>Component: {name}</p>
                <p>Movement Authority: PENDING</p>
            </body>
            </html>
        """))
    with open(os.path.join(path, 'Dockerfile'), 'w') as f:
        f.write(textwrap.dedent("""
            FROM nginx:alpine
            COPY driver.html /usr/share/nginx/html/index.html
        """))

# ──────────────────────────────────────────────
# T2 — Communication
# ──────────────────────────────────────────────

def generate_api_gateway(name):
    path = f'skeleton/{name}'
    os.makedirs(path, exist_ok=True)
    with open(os.path.join(path, 'app.py'), 'w') as f:
        f.write(textwrap.dedent(f"""
            from flask import Flask, request, jsonify
            import requests

            app = Flask(__name__)

            ROUTES = {{
                '/passengers': 'http://passengers-ms:80',
                '/routes':     'http://routes-ms:80',
                '/trains':     'http://trains-ms:80',
                '/position':   'http://position-time-ms:80',
                '/tickets':    'http://tickets-ms:80',
                '/authority':  'http://mas:80',
            }}

            @app.route('/<path:path>', methods=['GET', 'POST', 'PUT',
'DELETE'])
            def gateway(path):
                for prefix, target in ROUTES.items():
                    if ('/' + path).startswith(prefix):
                        url = target + '/' + path
                        resp = requests.request(
                            method=request.method,
                            url=url,
```

```
                            json=request.get_json(silent=True)
                        )
                        return jsonify(resp.json()), resp.status_code
                return jsonify(error='Route not found'), 404

            if __name__ == '__main__':
                app.run(host='0.0.0.0', port=80)
        """))
    with open(os.path.join(path, 'Dockerfile'), 'w') as f:
        f.write(textwrap.dedent("""
            FROM python:3.11-slim
            WORKDIR /app
            COPY . .
            RUN pip install flask requests
            CMD ["python", "app.py"]
        """))

# ──────────────────────────────────────────────
# T3 — Logic
# ──────────────────────────────────────────────

def generate_microservice(name, database=None):
    path = f'skeleton/{name}'
    os.makedirs(path, exist_ok=True)
    db_host = database if database else f'{name}-db'
    with open(os.path.join(path, 'app.py'), 'w') as f:
        f.write(textwrap.dedent(f"""
            from flask import Flask, jsonify, request
            import mysql.connector

            app = Flask(__name__)

            def get_conn():
                return mysql.connector.connect(
                    host='{db_host}',
                    user='root',
                    password='root',
                    database='{db_host}'
                )

            @app.route('/health')
            def health():
                return jsonify(status='ok', service='{name}')

            @app.route('/records')
            def get_records():
                conn = get_conn()
                cursor = conn.cursor()
                cursor.execute("SELECT * FROM records")
                rows = cursor.fetchall()
                cursor.close()
                conn.close()
                return jsonify(records=rows)
```

```
            @app.route('/records', methods=['POST'])
            def create_record():
                data = request.json
                conn = get_conn()
                cursor = conn.cursor()
                cursor.execute(
                    "INSERT INTO records (name) VALUES (%s)",
                    (data.get('name', 'unknown'),)
                )
                conn.commit()
                cursor.close()
                conn.close()
                return jsonify(status='created'), 201

            if __name__ == '__main__':
                app.run(host='0.0.0.0', port=80)
        """))
    with open(os.path.join(path, 'Dockerfile'), 'w') as f:
        f.write(textwrap.dedent("""
            FROM python:3.11-slim
            WORKDIR /app
            COPY . .
            RUN pip install flask mysql-connector-python
            CMD ["python", "app.py"]
        """))

def generate_authority_service(name):
    path = f'skeleton/{name}'
    os.makedirs(path, exist_ok=True)
    with open(os.path.join(path, 'app.py'), 'w') as f:
        f.write(textwrap.dedent(f"""
            from flask import Flask, jsonify, request

            app = Flask(__name__)

            @app.route('/health')
            def health():
                return jsonify(status='ok', service='{name}')

            @app.route('/authority', methods=['POST'])
            def request_authority():
                data = request.json
                train_id   = data.get('train_id')
                corridor   = data.get('corridor')
                # Placeholder: real logic would query position-time-ms and
routes-ms
                granted = True
                return jsonify(
                    train_id=train_id,
                    corridor=corridor,
                    movement_authority='GRANTED' if granted else 'DENIED'
                )

            if __name__ == '__main__':
```

```
                app.run(host='0.0.0.0', port=80)
        """))
    with open(os.path.join(path, 'Dockerfile'), 'w') as f:
        f.write(textwrap.dedent("""
            FROM python:3.11-slim
            WORKDIR /app
            COPY . .
            RUN pip install flask
            CMD ["python", "app.py"]
        """))

# ──────────────────────────────────────────────
# T4 — Data
# ──────────────────────────────────────────────

def generate_database(name):
    path = f'skeleton/{name}'
    os.makedirs(path, exist_ok=True)
    with open(os.path.join(path, 'init.sql'), 'w') as f:
        f.write(textwrap.dedent(f"""
            CREATE TABLE IF NOT EXISTS records (
                id   INT AUTO_INCREMENT PRIMARY KEY,
                name VARCHAR(255) NOT NULL
            );
        """))

def generate_data_lake(name):
    path = f'skeleton/{name}'
    os.makedirs(path, exist_ok=True)
    with open(os.path.join(path, 'README.md'), 'w') as f:
        f.write(textwrap.dedent(f"""
            # {name} — ERTMS Data Lake

            Aggregated operational and historical data lake.

            ## Schemas
            - operational/   : real-time ingestion from position-time-ms
            - historical/    : archived train movement records
            - audit/         : movement authority logs
        """))
    with open(os.path.join(path, 'ingest.py'), 'w') as f:
        f.write(textwrap.dedent(f"""
            # Placeholder ingestion script for the ERTMS Data Lake
({name})
            import json, datetime

            def ingest(event: dict):
                record = {{
                    'timestamp': datetime.datetime.utcnow().isoformat(),
                    'payload': event
                }}
                print(json.dumps(record))

            if __name__ == '__main__':
```

```
                ingest({{'train_id': 'T-001', 'position':
'48.8566,2.3522', 'speed_kmh': 220}})
        """))

# ──────────────────────────────────────────────
# T5 — Physical
# ──────────────────────────────────────────────

def generate_onboard_unit(name):
    path = f'skeleton/{name}'
    os.makedirs(path, exist_ok=True)
    with open(os.path.join(path, 'obu.py'), 'w') as f:
        f.write(textwrap.dedent(f"""
            # Onboard Unit ({name}) — ERTMS T5
            # Simulates the main computing unit aboard the train.

            import time, random

            def read_sensors():
                return {{
                    'speed_kmh': round(random.uniform(0, 300), 1),
                    'position':  f'{{random.uniform(40, 55):.4f}},
{{random.uniform(-5, 25):.4f}}'
                }}

            def send_to_ground(data):
                print(f'[OBU {name}] Sending to ground control: {{data}}')

            def receive_command(cmd):
                print(f'[OBU {name}] Command received: {{cmd}}')
                if cmd.get('action') == 'BRAKE':
                    apply_brake(cmd.get('intensity', 1.0))

            def apply_brake(intensity):
                print(f'[OBU {name}] Applying brake at intensity
{{intensity}}')

            if __name__ == '__main__':
                while True:
                    data = read_sensors()
                    send_to_ground(data)
                    time.sleep(1)
        """))

def generate_sensor(name):
    path = f'skeleton/{name}'
    os.makedirs(path, exist_ok=True)
    with open(os.path.join(path, 'sensor.py'), 'w') as f:
        f.write(textwrap.dedent(f"""
            # Train Sensor ({name}) — ERTMS T5
            # Simulates speed, position, and physical condition detection.

            import time, random
```

```
            def read():
                return {{
                    'speed_kmh':   round(random.uniform(0, 300), 1),
                    'position':    f'{{random.uniform(40, 55):.4f}},
{{random.uniform(-5, 25):.4f}}',
                    'door_closed': random.choice([True, True, True,
False]),
                    'pantograph':  'UP'
                }}

            if __name__ == '__main__':
                while True:
                    print(f'[SENSOR {name}]', read())
                    time.sleep(0.5)
        """))

def generate_actuator(name):
    path = f'skeleton/{name}'
    os.makedirs(path, exist_ok=True)
    with open(os.path.join(path, 'actuator.py'), 'w') as f:
        f.write(textwrap.dedent(f"""
            # Brake Actuator ({name}) — ERTMS T5
            # Receives control commands and acts on the physical train.

            def execute(command):
                action    = command.get('action')
                intensity = command.get('intensity', 1.0)
                if action == 'BRAKE':
                    print(f'[ACTUATOR {name}] Emergency brake — intensity
{{intensity}}')
                elif action == 'RELEASE':
                    print(f'[ACTUATOR {name}] Brake released')
                else:
                    print(f'[ACTUATOR {name}] Unknown command:
{{action}}')

            if __name__ == '__main__':
                execute({{'action': 'BRAKE', 'intensity': 0.8}})
        """))

def generate_balise(name):
    path = f'skeleton/{name}'
    os.makedirs(path, exist_ok=True)
    with open(os.path.join(path, 'balise.py'), 'w') as f:
        f.write(textwrap.dedent(f"""
            # Balise ({name}) — ERTMS T5
            # Track-side transponder transmitting position reference data
to passing trains.

            BALISE_ID  = '{name}'
            POSITION_M = 12500   # metres from reference point

            def transmit():
                return {{
```

```
                    'balise_id':   BALISE_ID,
                    'position_m':  POSITION_M,
                    'track_id':    'CORRIDOR-A',
                    'signal':      'EUROBALISE-STM'
                }}

            if __name__ == '__main__':
                print('[BALISE]', transmit())
        """))

# ──────────────────────────────────────────────
# docker-compose
# ──────────────────────────────────────────────

TIER_ORDER = {
    'data':         0,
    'physical':     1,
    'logic':        2,
    'communication':3,
    'presentation': 4,
}

DB_IMAGES = {'database', 'data_lake'}

def generate_docker_compose(components):
    """components: { name: (tier, type) }"""
    path = 'skeleton/'
    os.makedirs(path, exist_ok=True)

    sorted_items = sorted(
        components.items(),
        key=lambda kv: TIER_ORDER.get(kv[1][0], 5)
    )

    db_names = [n for n, (t, ct) in sorted_items if ct == 'database']

    with open(os.path.join(path, 'docker-compose.yml'), 'w') as f:
        f.write("services:\n")
        for i, (name, (tier, ctype)) in enumerate(sorted_items):
            port = 8000 + i
            f.write(f"  {name}:\n")

            if ctype == 'database':
                f.write("    image: mysql:8\n")
                f.write("    environment:\n")
                f.write("      - MYSQL_ROOT_PASSWORD=root\n")
                f.write(f"      - MYSQL_DATABASE={name}\n")
                f.write("    volumes:\n")
                f.write(f"      - ./{name}/init.sql:/docker-entrypoint-
initdb.d/init.sql\n")
                f.write("    ports:\n")
                f.write("      - '3306:3306'\n")

            elif ctype == 'data_lake':
```

```
                f.write("    image: python:3.11-slim\n")
                f.write(f"    volumes:\n      - ./{name}:/app\n")
                f.write("    working_dir: /app\n")
                f.write("    command: python ingest.py\n")

            elif ctype in {'onboard_unit', 'sensor', 'actuator',
'balise'}:
                f.write("    image: python:3.11-slim\n")
                script = {
                    'onboard_unit': 'obu.py',
                    'sensor':       'sensor.py',
                    'actuator':     'actuator.py',
                    'balise':       'balise.py',
                }[ctype]
                f.write(f"    volumes:\n      - ./{name}:/app\n")
                f.write("    working_dir: /app\n")
                f.write(f"    command: python {script}\n")

            else:
                # microservice, api_gateway, authority_service, web UIs
                f.write(f"    build: ./{name}\n")
                f.write(f"    ports:\n      - '{port}:80'\n")
                if ctype == 'microservice' and db_names:
                    f.write(f"    depends_on:\n")
                    for db in db_names:
                        f.write(f"      - {db}\n")

        f.write("\nnetworks:\n  default:\n    driver: bridge\n")


# ────────────────────────────────────────────
# Dispatcher
# ────────────────────────────────────────────

GENERATORS = {
    'web_interface':    generate_web_interface,
    'operator_ui':      generate_operator_ui,
    'driver_ui':        generate_driver_ui,
    'api_gateway':      generate_api_gateway,
    'microservice':     generate_microservice,
    'authority_service':generate_authority_service,
    'database':         generate_database,
    'data_lake':        generate_data_lake,
    'onboard_unit':     generate_onboard_unit,
    'sensor':           generate_sensor,
    'actuator':         generate_actuator,
    'balise':           generate_balise,
}

def apply_transformations(model):
    components = {}   # name -> (tier, type)
    db_map     = {}   # microservice_name -> database_name (heuristic:
shared prefix)
```

```
    # First pass: collect all components
    for e in model.elements:
        if e.__class__.__name__ == 'Component':
            components[e.name] = (e.tier, e.type)

    # Second pass: link microservices to their databases by name prefix
    db_names = [n for n, (t, ct) in components.items() if ct ==
'database']
    for name, (tier, ctype) in components.items():
        if ctype == 'microservice':
            for db in db_names:
                if db.replace('-db', '').replace('_db', '') in name:
                    db_map[name] = db
                    break

    # Generate skeletons
    for name, (tier, ctype) in components.items():
        gen = GENERATORS.get(ctype)
        if gen:
            if ctype == 'microservice':
                gen(name, database=db_map.get(name))
            else:
                gen(name)

    generate_docker_compose(components)
```

3.3. Modeling

Create a **model.arch** file representing the ERTMS architecture across its five tiers:

```
architecture:

    // T1 — Presentation
    component presentation web_interface    portal
    component presentation operator_ui      dashboard
    component presentation driver_ui        driver_interface

    // T2 — Communication
    component communication api_gateway     api_gateway

    // T3 — Logic
    component logic microservice            passengers_ms
    component logic microservice            routes_ms
    component logic microservice            trains_ms
    component logic microservice            position_time_ms
    component logic microservice            tickets_ms
    component logic authority_service       mas

    // T4 — Data
    component data database                 passengers_db
```

```
    component data database                 routes_db
    component data database                 trains_db
    component data database                 position_time_db
    component data data_lake                dl

    // T5 — Physical
    component physical onboard_unit         onboard_unit
    component physical sensor               train_sensor
    component physical actuator             brake_actuator
    component physical balise               balise

    // Connectors — T1 → T2
    connector data_flow         portal              -> api_gateway
    connector data_flow         dashboard           -> api_gateway
    connector data_flow         driver_interface    -> api_gateway

    // Connectors — T2 → T3
    connector data_flow         api_gateway         -> passengers_ms
    connector data_flow         api_gateway         -> routes_ms
    connector data_flow         api_gateway         -> trains_ms
    connector data_flow         api_gateway         -> position_time_ms
    connector data_flow         api_gateway         -> tickets_ms
    connector data_flow         api_gateway         -> mas

    // Connectors — T3 → T4
    connector dependency        passengers_ms       -> passengers_db
    connector dependency        routes_ms           -> routes_db
    connector dependency        trains_ms           -> trains_db
    connector dependency        position_time_ms    -> position_time_db
    connector data_flow         position_time_ms    -> dl

    // Connectors — T3 intra-tier
    connector data_flow         mas                 -> position_time_ms
    connector data_flow         mas                 -> routes_ms

    // Connectors — T5 → T3
    connector event_notification train_sensor       -> onboard_unit
    connector data_flow          onboard_unit       -> position_time_ms

    // Connectors — T3 → T5
    connector control_command    mas                -> onboard_unit
    connector control_command    onboard_unit       -> brake_actuator

    // Connectors — T5 intra-tier
    connector data_flow          balise             -> onboard_unit
```

3.4. Generation

Create a **generation.py** file to execute the generation process:

```python
from metamodel import create_metamodel
from transformations import apply_transformations

if __name__ == '__main__':
    metamodel = create_metamodel()
    model = metamodel.model_from_file('model.arch')
    apply_transformations(model)
```

3.5. Testing

a. Create a **Dockerfile** to specify the requirements to execute the program:

```dockerfile
FROM python:3.11-slim

WORKDIR /app

COPY . .

RUN pip install --no-cache-dir textX mysql-connector-python flask

CMD ["python", "generation.py"]
```

b. Build the Docker image:

```bash
docker build -t lssa-lab2 .
```

c. Run the container to generate the ERTMS skeleton:

```bash
docker run --rm -v "$PWD:/app" lssa-lab2
```

After running the above command, a `skeleton/` directory is created containing one subdirectory per component, organized by tier.

d. Enter the `skeleton/` directory:

```bash
cd skeleton
```

e. Launch the generated skeleton:

```bash
docker-compose up --build
```

f. Verify the running containers:

```
docker ps -a
```

You should see containers for the API Gateway, all microservices, MAS, and the MySQL databases. Physical-tier components (Onboard Unit, Sensor, Actuator, Balise) run as Python-only containers that print simulated data to stdout.

g. Open the Operator Dashboard in a web browser: `http://localhost:8003`

h. Verify the API Gateway routing by calling a microservice through it:

```
curl http://localhost:8002/passengers/records
```

i. Verify the Movement Authority Service:

```
curl -X POST http://localhost:8002/authority \
     -H "Content-Type: application/json" \
     -d '{"train_id": "T-001", "corridor": "CORRIDOR-A"}'
```

j. Inspect the database for a microservice directly:

```
docker exec -it skeleton-passengers_db-1 sh
```

```
mysql -u root -p
```

Password = *root*

```
USE passengers_db;
SELECT * FROM records;
```

---

4. Delivery

The delivery consists of two artifacts:

1. The **source files** of the MDE program (`arch.tx`, `metamodel.py`, `transformations.py`, `model.arch`, `generation.py`, `Dockerfile`) and the **generated `skeleton/`** directory.
2. A **brief written report** (≤ 1 page, `report.pdf`) answering the following:

- Describe the process of translating the C&C view from Lab 1 into a DSL model. What was straightforward? What required the most judgment or adaptation?
- What limitations did you encounter in the DSL grammar or the transformation rules when representing the ERTMS? How did you work around them?

4.1. Submission Format

The delivery must be submitted via Google Classroom. File in *.zip* format named **lab2-team-X**, where X = team (1a, 1b, ... 2a, 2b, ...).

4.2. Delivery Deadline

Friday, April 10, 2026, before 23h59.